

Capitolul 5. Modelarea traficului în Linux

Obiectivul lucrării

Lucrarea arată modul în care putem controla traficul care trece printr-un echipament de rețea în scopul controlului calității serviciului sau pentru testarea aplicațiilor în condiții diferite de trafic.

Kerneul Linux permite funcții avansate de control și modelare a traficului prin folosirea comenzii tc (Traffic Control). Modelarea traficului se poate realiza doar pentru protocoalele care au suport în acest sens. De exemplu protocolul TCP implementează un mecanism prin care crește fluxul de date după fiecare transmisie reușită până când ocupa toată lățimea de bandă (sau este limitat de sursă). În cazul în care unele pachete sunt pierdute, fluxul de date este redus. Prin urmare simplă eliminare a pachetelor la intrarea în ruter permite controlul traficului pentru acest protocol. Alte protocoale însă, precum UDP, nu sunt sensibile la pierderile de pachete și acest mecanism nu poate fi folosit.

Breviar teoretic

Latenta în rețelele de calculatoare este o componentă importantă a calității serviciului deoarece exprimă timpul necesar pachetelor pentru a ajunge de la sursă la destinație. Cu cât latența este mai mare cu atât calitatea serviciului scade. Anumite servicii de rețea sunt mai sensibile la creșterea latenței sau la variația latenței decât altele.

Componentele latenței în rețele de calculatoare

Considerăm P ca fiind lungimea unui pachet exprimată în biți, L lungimea traseului de date exprimată în metri și R rata de transfer exprimată în biți pe secunda. Atunci putem defini:

- **întârzierea de propagare** ca fiind timpul necesar unui bit pentru a parcurge lungimea traseului (L) de comunicație, circulând cu viteza c (viteza de propagare prin mediul de transmisie):

$$\text{PROP} = \frac{L}{c}$$

De exemplu întârzierea de propagare prin liniile de cupru (cablu torsadat sau cablu coaxial) este de 4.76 microsecunde pe kilometru. Deoarece liniile de comunicație sunt foarte lungi, valoarea acestei întârzieri este de ordinul milisecundelor.

- **întârzierea de transmisie** este timpul necesar pentru a transmite un pachet de lungime P. Acest timp apare deoarece transmisiile în rețele de calculatoare sunt seriale și este necesar un anumit timp pentru a transfera datele din bufferul de transmisie în linia de comunicație (respectiv preluarea datelor din linie în bufferul de recepție). De exemplu pentru o linie de comunicație Ethernet de 1Gbps durează 10 microsecunde pentru a recepționa 10000 de biți.

$$\text{TRANSM} = \frac{P}{R}, \text{ unde } R \text{ este rata de transmisie (b/s)}$$

Într-o rețea ideală latența totală este prin urmare suma celor doi timpi:

$$\text{Latența} = \text{PROP} + \text{TRANSM};$$

Reprezentând grafic aceste valori se obține graficul următor:

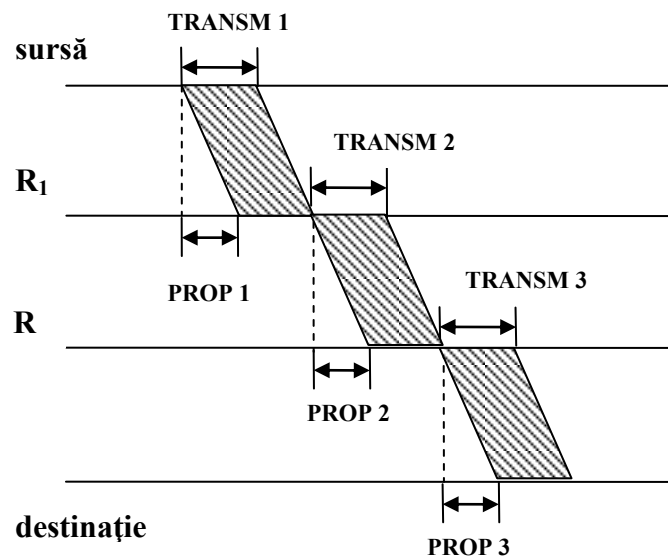


Figura 1 Componentele latenței datorate întârzierii de propagare și transmisie

Prin urmare valoarea minimă a latenței pentru transmiterea tuturor pachetelor este:

$$\text{Latența} = \sum_{i=0}^n (\text{TRANSM}_i + \text{PROP}_i)$$

Din păcate în rețelele reale echipamentele intermediare (rutere, switch-uri) nu procesează datele instantaneu, fiecare având o coadă de așteptare variabilă care este procesată secvențial. Suplimentar, orice echipament are o întârziere implicită, datorată traversării datelor de la intrarea în echipament până la ieșirea din echipament, plus timpul de căutare a în tabela de adrese.

Întârzierea depinde la switch-uri de arhitectura acestuia (cut-through sau store-and-forward) și este mult mai mică decât la rutere.

La rutere într-un interval de timp scurt sau chiar în același timp la un echipament pot sosi mai multe pachete simultan, fiecare având destinații diferite. Echipamentele care direcționează

traficul trebuie sa calculeze pentru fiecare pachet destinația însă, deoarece calculul nu se poate efectua simultan, unele vor fi procesate mai rapid și altele cu o anumită întârziere. Datele care urmează a fi procesate vor fi stocate temporar într-o structura de tip coada. Întârzierea datorată cozii de așteptare Q este singura variabilă în rețeaua de calculatoare.

Această componentă face ca latența să nu fie compusă doar din componente de valoare constantă. Prin urmare, notând cu Q_i întârzierea datorată cozii de așteptare formula de calcul a latentei se modifică în modul următor:

$$\text{Latenta} = \sum_{i=0}^n (\text{TRANSP}_i + \text{PROP}_i + Q_i)$$

Etapale parcurse de un pachet în traversarea unui echipament de rețea cu funcții de calitate a serviciului

În rețelele de calculatoare traficul care traversează un ruter ce oferă funcția de calitate a serviciului parcurge etapele de CQS (Classification Queing Scheduling):

- clasificare: pachetele recepționate vor fi direcționate către una din clasele disponibile în funcție de caracteristicile sale;
- plasarea în coada de așteptare: pachetele vor rămâne în coada de așteptare până când vor fi scoase de un planificator.
- planificarea pentru ieșirea în rețea: planificatorul determină care pachet va fi extras din coada (sau cozile) de așteptare pentru a fi trimis în rețea. Trebuie să fie cât mai simplă, dar totuși să îndeplinească condițiile solicitate: latența, variația latenței, etc.

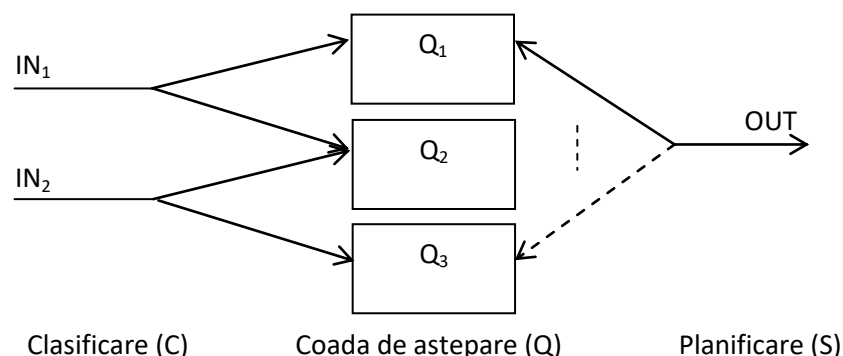


Figura 2 Etapele pe care le parcurge un pachet la traversarea unui echipament de rețea cu funcții de calitate a serviciului

Modelarea traficului (traffic shaping) este un alt mecanism pe care aceste echipamente de rețea îl pot implementa, prin care pachetele sunt întârziate pentru a îndeplini cerințe legate de rata de ieșire. Deoarece acest mecanism limitează traficul, el are ca efect secundar reducerea vârfurilor de trafic care nu sunt de dorit într-o rețea deoarece pot încălca dincolo de posibilitățile de procesare echipamentele de rețea (și pot genera creșterea excesivă a latentei sau pierderi de pachete).

Traffic Control (tc)

Utilizatul Traffic Control permite: modelarea, planificarea ieșirilor, politica aplicată intrărilor și eliminarea traficului.

qdisc este termenul folosit pentru disciplina de punere în coada de așteptare („queuing discipline”) și este o componentă fundamentală a controlului de trafic. Când kernelul sistemului de operare dorește să trimită un pachet, el este plasat în *qdisc*-ul asociat acelei interfețe. În continuare kernelul încearcă să scoată cât mai multe pachete din *qdisc*, pentru a fi trimise prin adaptorul de rețea.

În mod implicit este setată politica *fifo_fast*.

Există două tipuri de discipline de punere în coada de așteptare:

- cu clase: Aceste discipline permit crearea claselor de filtrare a traficului. Fiecare clasă are propriul set de reguli pentru filtrarea traficului. În plus, fiecare clasă poate avea asignată alte discipline de plasare în coada de pachete, cu sau fără clasă.

- fără clase: Nu permit adăugarea de mai multe discipline de plasare în coada de pachete.

Comanda care șterge orice disciplină de plasare în coada de pachete setată pentru interfața *eth0* este următoarea:

```
tc qdisc del root dev eth0
```

Pentru a adăuga o disciplină de plasare în coada pentru *eth0* se folosește:

```
tc qdisc add dev eth0 root QDISC ParametriQDISC
```

În continuare vor fi prezentate cele mai folosite tehnici pentru disciplina de punere în coada de așteptare.

Discipline de plasare în coada de pachete fără clase

Aceste cozi permit managementul simplu al traficului prin reordonare, încetinind sau eliminând pachete. Ele nu permit crearea de clase de trafic.

pfifo

Este cel mai simplu qdisc, care implementează mecanismul FIFO. Permite limitarea traficului prin numărul de pachete sau numărul de octeți.

pfifo_fast

Este politica utilizată în mod implicit pe root pentru dispozitivele care nu au fost configurate. Algoritmul folosit este First In First Out, pachetele de date fiind procesate în ordinea sosirii. Deși nu se efectuează o procesare, coada permite păstrarea temporară a pachetelor dacă placa de rețea nu le poate prelua momentan.

Random Early Detection (red)

Random Early Detection simulează congestia în rețea prin eliminarea de pachete în mod aleatoriu atunci când traficul se pregătește să depășească lățimea de banda maximă alocată. Fata de cozile simple care doar elimina pachetele în cazul în care coada de așteptare este plină, RED realizează eliminarea pachetelor cu mult înainte de a se atinge lățimea de banda maximă. În acest mod retransmișile datorate pachetelor picate nu se vor realiza toate în același timp ci în mod treptat, încărcând treptat, în timp, rețeaua. Cu cât numărul de pachete în coada de așteptare este mai mare, cu atât probabilitatea de eliminare a unui pachet este mai mare.

Câteva din filtrele ce pot fi utilizate: min – dimensiunea medie a cozii pentru care marcarea pachetelor pentru eliminare devine posibilă; max – dimensiunea medie a cozii pentru care probabilitatea de marcarea este maximă; probability – probabilitatea maximă de marcarea exprimată de un număr între 0.0 și 1.0; burst – folosită în determinarea vitezei cu care calculul a valorii medii a cozii, ia în calcul valoarea reală a cozii de pachete (o valoare mare face ca algoritmul RED să constate modificarea valorii medii mai lent și să înceapă marcarea pachetelor mai târziu); avpkt – valoare specificată în octeți, folosită împreună cu burst pentru a determina constanta de timp pentru calculul dimensiunii medii a cozii;

```
tc qdisc add dev eth0 root red limit 4000 min 300 max 900 probability 0.2 avpkt 10 burst
```

Token Bucket Filter (tbf)

Aceasta politica permite trecerea pachetelor dacă nu a fost depășit un nivel limitat stabilit.

Algoritmul Token Bucket are rolul doar de a modela traficul și nu realizează o planificare a acestuia. În acest algoritm se folosesc jetoane pentru a controla fluxul de pachete astfel:

- pentru fiecare jeton generat, un pachet pleacă pe o interfață;
- jetoanele se generează în mod constant în funcție de regulile configurare. Numărul maxim de jetoane care pot fi generate dacă nu sunt pachete disponibile este egal cu traficul maxim care este ieși într-o singura transmisie;
- dacă nu sunt jetoane disponibile, pachetele sunt adăugate într-o coada de așteptare, pana la limita maxima configurata. Dacă în acest caz sosesc jetoane, pachetele vor ieși imediat producând pentru perioade scurte de timp trafic de nivel foarte mare. Această a doua situație nu este de dorit. Dacă jetoanele nu sosesc suficient de rapid, pachetele care sunt în așteptarea unui jeton, vor fi eliminate;

Parametri folosiți:

- limit este numărul de octeți care pot fi plasați în coada în așteptarea jetoanelor. Este folosita complementar comenzii latency.
- latency specifica timpul maxim pe care un pachet poate sa îl petreacă în coda de așteptare. Este folosita complementar comenzii limit.
- burst este numărul de octeți maxim care pot ieși într-o singura transmisie. Dimensiunea pentru burst trebuie corelata cu lățimea de banda maximă. De exemplu pentru 10Mbps trebuie alocati 10kB pentru buffer. Dacă bufferul este prea mic, viteza maxima de transfer nu va putea fi atinsa.

Următoarea comanda realizează pentru dispozitivul eth0 o limitare a vitezei de 250kbps, pachetele pot sta în coada de așteptare 100ms înainte de a fi eliminate și un maxim de trafic de 1,5kbps.

```
tc qdisc add dev eth0 root tbf rate 250kbit latency 100ms burst 1500
```

Stochastic Fairness Queueing (SFQ)

Această disciplină de plasare în coada de pachete grupează conversațiile în fluxuri („hash bucket”) și fiecare flux are, pe rând, posibilitatea transmiterii date conform unui algoritm Round-Robin. În acest mod fiecare flux are posibilitatea de a transmite date și previne situația în care un flux ar transmite semnificativ mai multe date decât celelalte fluxuri.

Clasificarea în fluxuri se face în urma calculării unui hash pe baza adresei sursă, a destinației și a portului sursă. Deoarece uneori fluxuri multiple pot genera același hash și în acest mod vor fi grupate incorect, este necesara stabilirea unui interval la care se vor recalcula (numit perturb) hash-urile. În acest mod gruparea incorectă durează foarte puțin timp.

În cazul în care numărul de pachete depășește lungimea maximă a cozii de așteptare, SFQ elimina pachete din fluxul care are cele mai multe pachete în așteptare. SFQ devine util doar în cazul în care interfața de transmisie este complet ocupată altfel nu există cozi de așteptare și algoritmul nu intră în acțiune.

În următorul exemplu se setează pentru interfața eth0 disciplina SFQ și se stabilește un interval de 10 secunde la care să se recalculeze hash-ul pentru a evita grupare incorectă:

```
tc qdisc add dev eth0 root sfq perturb 10
```

Discipline de plasare în coada de pachete cu clase

Disciplinele de plasare în coada de pachete cu clase sunt necesare dacă există mai multe tipuri de trafic pe care dorim să le tratăm diferit. Fiecare clasă de trafic creată are nevoie de un nume dar și de un identificator pentru părintele clasei. Părintele poate fi clasa rădăcina (root) a unei interfețe sau o altă clasă.

Numele vor fi setate cu *nume_radacina:nume_clasa* în care *nume_radacina* este de obicei 1 iar numele clasei este cel dorit.

Hierarchical Token Bucket (HTB)

Această disciplină de plasare în coada de pachete este potrivită pentru situațiile în care lățimea de bandă trebuie împărțită între mai multe tipuri de trafic, fiecare având specificată lățimea de bandă garantată. HTB modelează traficul folosind algoritmul Token Bucket.

Într-o instanță HTB pot exista mai multe clase, fiecare dintre ele putând fi împărțită în mai multe sub-clase. În mod implicit se folosește pfifo.

Când plasează un pachet în coada de așteptare, HTB începe de la root și determină în continuare de la nod la nod care clasa ar trebui să recepționeze datele. Dacă clasa nu are alte sub-clase, pachetul este adăugat la acea coadă de așteptare. În cazul în care sunt sub-clase, se continuă procesul de căutare.

Configurarea HTB pentru interfața eth0 urmează următorii pași:

1. se șterge metoda PFIFO_FAST dacă acesta este configurat:

```
tc qdisc del dev eth0 root
```

2. se adaugă disciplina HTB la nivelul root (1):

```
tc qdisc add dev eth0 root handle 1: htb
```

Suplimentar poate fi precizata o clasa care va fi folosita în mod implicit pentru pachete cu ajutorul comenzii:

```
tc qdisc add dev eth0 root handle 1: htb default 20
```

3. Se creează clasele 10 și 20 în care vor fi plasate pachetele și se specifica pentru fiecare clasa rata dorita de ieșire:

```
tc class add dev eth0 parent 1: classid 1:10 htb 200kbit
```

```
tc class add dev eth0 parent 1: classid 1:20 htb 1mbit
```

4. Se adaugă filtrele care vor selecta care pachete vor ajunge în clasa 10 și care pachete vor ajunge în clasa 20. Formatul general este:

```
tc filter add dev eth0 protocol ip parent 1: prio 1 u32 match ip dst [Adresa IP]/[Masca] flowid 1:[Id flux]
```

În cazul următor este folosita adresa sursă a pachetului (src) pentru a determina unde va fi plasat pachetul:

```
tc filter add dev eth0 protocol ip parent 1: prio 1 u32 match ip src 10.0.0.1/255.0.0.0  
match ip dport 80 flowid 1:10
```

```
tc filter add dev eth0 protocol ip parent 1: prio 2 u32 match ip dst 10.0.0.1/255.0.0.0  
match ip dport 22 flowid 1:20
```

Vizualizarea disciplinei setate se face cu:

```
tc filter show dev eth0
```

Aplicații

Network Emulation (netem)

Utilitarul netem asigură funcționalitatea necesară pentru a testa protocoalele de rețea prin emularea proprietăților unei rețele. Se asigură suportul pentru a emula:

- introducerea latentei în traficul de rețea;
- variația întârzierii (jitter) în mod variabil sau constant;
- pierderea de pachete;
- duplicarea pachetelor;
- reordonarea pachetelor.

Netem este configurat cu ajutorul utilitarul în linie de comandă tc.

Introducerea latentei în traficul de rețea

Latenta are următoarele componente:

- întârzierea de propagare;
- întârzierea de transmitere;
- întârzierea datorată cozii de procesare a echipamentelor de rețea (în special ruterele).

Pentru a adăuga o întârziere se folosește comanda:

```
tc qdisc add dev eth0 root netem delay 200ms
```

Variația întârzierii (jitter)

Variația întârzierii este un factor important de influență a calității comunicațiilor de voce. Dacă se dorește ca întârzierea introdusă să se apropie de cea a unei rețele reale, se poate folosi comanda:

```
tc qdisc change dev eth0 root netem delay 100ms 10ms
```

Această comandă introduce o întârziere de 100ms cu o variație de +/- 10ms.

Uneori variațiile latentei dintr-o rețea nu sunt complet aleatorii. Când ruterele devin încărcate datorită numărului mare de conexiuni sau a traficului susținut în rețea, ele introduc perioade mai lungi în care variațiile latentei cresc sau scad puțin în jurul unor valori. Pentru a emula acest comportament se poate adăuga pentru fiecare pachet o întârziere a cărei variație să difere de valoarea anterioară doar într-un anumit procent. Această variație poate fi simulată cu ajutorul comenzii:

```
tc qdisc change dev eth0 root netem delay 100ms 10ms 25%
```

Pierderea de pachete

Comunicațiile prin rețele de calculatoare au uneori pierderi de pachete, datorate echipamentelor defecte, coliziunilor cu alte pachete, mediului de transmisie care introduce zgomot în semnalul transmis cât și din alte cauze. Este important să analizăm ce se întâmplă în aceste situații cu aplicațiile pe care le folosim.

Pentru a simula pierderea de pachete se folosește comanda:

```
tc qdisc change dev eth0 root netem loss 0.2%
```

Aici 2 din 1000 pachete este eliminat în mod aleatoriu. Similar cu situația anterioară, se poate preciza suplimentar o marjă de variație a pierderii de pachete, în acest mod se pot simula pierderile grupate de pachete.

```
tc qdisc change dev eth0 root netem loss 0.3% 25%
```

Duplicarea pachetelor

Dacă un protocol de rețea decide ca un pachet este pierdut și trebuie retransmis, dar acesta este în realitate doar întârziat în mod excesiv, este posibil ca pachete duplicate să ajungă la destinație. Deoarece aceasta situație care nu este de dorit poate să apară destul de rar, pentru a o simula cu netem se introduce:

```
tc qdisc change dev eth0 root netem duplicate 1%
```

Reordonarea pachetelor

Atunci când circula pe internet, pachetele sunt direcționate (rutate) individual în funcție de condițiile din rețea. Este prin urmare posibil ca pachetele să nu ajungă la destinație urmând același traseu deci pot să ajungă în ordine diferită de cea în care au fost trimise. Acest comportament poate fi simulat cu netem prin comanda:

```
tc qdisc change dev eth0 root netem gap 5 delay 10ms
```

Această comandă permite trecerea unui singur pachet din 5 fără întârziere, celelalte 4 având o întârziere de 10ms. În acest mod ordinea în care pachetele ajung la destinație va fi modificată.

Desfășurarea lucrării

1. Se va studia breviarul teoretic.
2. Se va construi o rețea de forma:

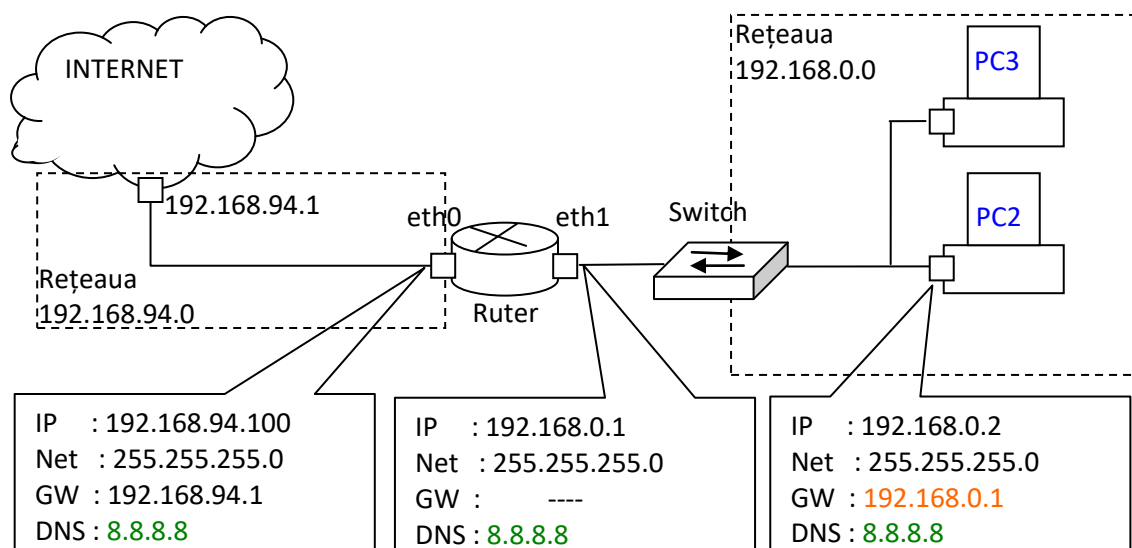


Figura 3 Structura rețelei de test pentru modelarea traficului

Ruterul din aceasta rețea va fi implementat în Linux și se vor testa pe acesta diferiți algoritmi de planificare a mesajelor. Configurația prezentată permite selecția traficului și în funcție de adresa IP.

3. Se va folosi aplicația ping pentru a testa efectele setărilor efectuate pe ruter care vizează traficul pe calculatoarele PC1 și PC2. Traficul va fi monitorizat în Windows cu *TaskManager* → *Performance* → *Open Resource Monitor* pentru tabul Network, iar în Linux cu utilitarul de consola *iptraf*.

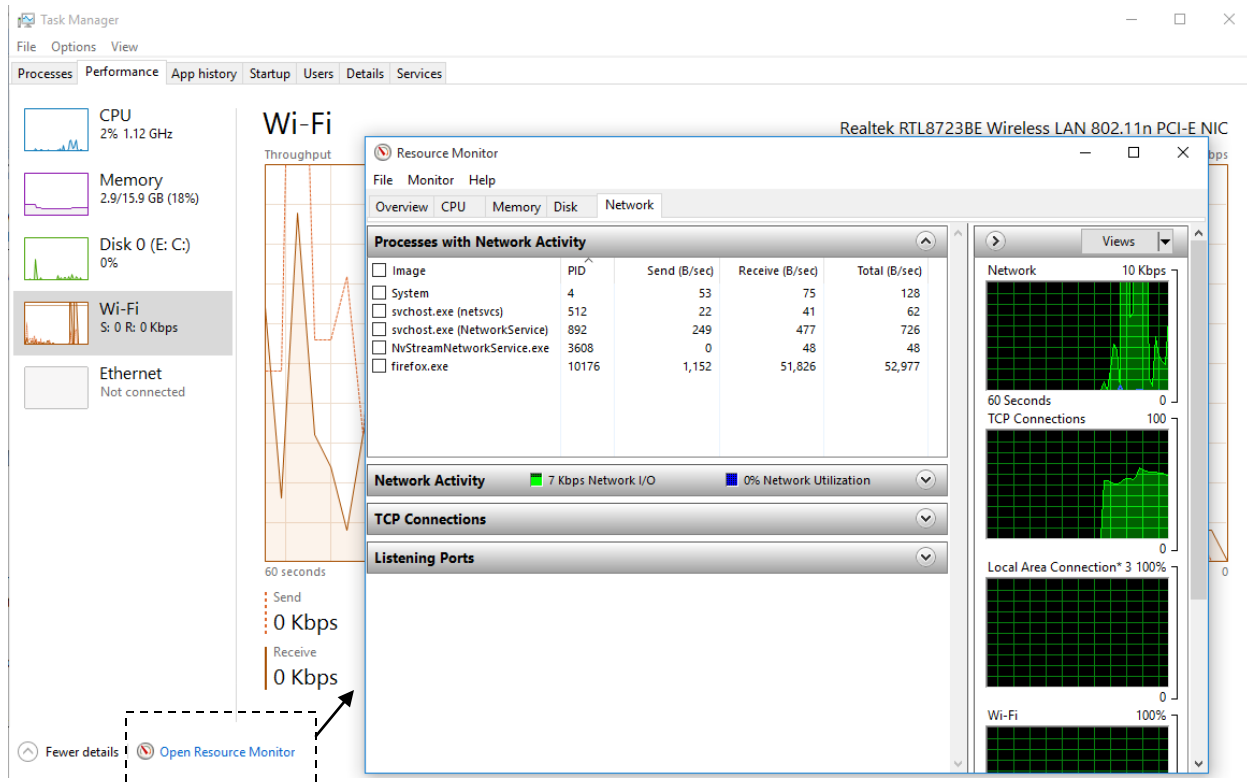


Figura 4 Monitorizarea traficului în Windows din Resource Monitor

4. Se va folosi aplicația Wireshark pentru a monitoriza schimbul de pachete din rețea.